Escuela Politécnica Nacional
Facultad de Ingeniería de Sistemas
"Aplicaciones Móviles"

Arquitectura y Evolución del Desarrollo Móvil

Integrantes: Priscila Pazmiño, Claudio Peñaherrera, Saul Tualombo

Fecha: 15/04/2026

Grupo: N°10

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

1. Desarrollo Nativo Puro (iOS/Android)

¿Qué es?

El desarrollo nativo implica construir aplicaciones específicas para cada sistema operativo utilizando sus SDKs oficiales, donde el código se compila directamente a binarios ejecutables optimizados para el hardware del dispositivo.

Tecnologías

- **Lenguajes de programación**
  - iOS: Swift, Objective-C
  - Android: Kotlin, Java
- **SDK (Software Development Kit)**
  - iOS SDK
  - Android SDK

    (Permiten acceder a hardware y servicios del sistema)
- **Frameworks de interfaz (UI)**
  - iOS: UIKit, SwiftUI
  - Android: Jetpack Compose, Android Views
- **Entornos de desarrollo (IDE)**
  - Xcode (iOS)
  - Android Studio (Android)
- **APIs y servicios del sistema**
  - Cámara
  - GPS
  - Notificaciones
  - Sensores

    - Red y almacenamiento
- **Herramientas de testing**
    - JUnit
    - Espresso
    - XCTest
- **Distribución de aplicaciones**
    - App Store
    - Google Play Store

Android Developers: https://developer.android.com

Apple Developer: https://developer.apple.com

Arquitectura

El desarrollo nativo utiliza una arquitectura en capas, donde la aplicación funciona directamente sobre el sistema operativo.

- **Presentación (UI):** interfaz gráfica (SwiftUI, Jetpack Compose)
- **Lógica de negocio:** procesamiento y reglas (Swift, Kotlin)
- **Datos:** manejo de APIs y bases de datos (Room, Core Data)
- **Sistema operativo:** acceso a hardware (cámara, GPS, sensores)

Se emplean patrones como **MVC, MVVM y Clean Architecture** para organizar el código y mejorar su mantenimiento.

Implicaciones ingenieriles

1. **Especialización tecnológica**
   Requiere uso de lenguajes específicos (Swift, Kotlin)

Necesita equipos separados por plataforma
Difícil reutilización de código

2. **Costo de desarrollo**
   Se deben crear dos aplicaciones (iOS y Android)
   Mayor inversión en tiempo y recursos
   Mantenimiento duplicado
3. **Rendimiento**
   Alto rendimiento al ejecutarse directamente en el sistema
   Mejor uso de CPU, memoria y batería
   Ideal para apps complejas o en tiempo real
4. **Seguridad**
   Acceso a mecanismos nativos de seguridad
   Mejor control de permisos
   Integración con biometría y hardware seguro
5. **Complejidad arquitectónica**
   Se manejan arquitecturas separadas (MVC, MVVM)
   Mayor esfuerzo en diseño y organización del sistema
6. **Ciclo de desarrollo**
   Cambios deben implementarse en ambas plataformas
   Mayor tiempo de desarrollo
   Riesgo de inconsistencias
7. **Experiencia de usuario (UX)**
   Interfaces más fluidas y naturales
   Mejor adaptación a cada sistema operativo
   Mayor satisfacción del usuario

Ventajas

1. **Rendimiento**
   Se ejecuta directamente sobre el sistema operativo
   Mayor velocidad y mejor uso de recursos
2. **Experiencia de usuario (UX/UI)**
   Uso de componentes nativos (SwiftUI, Jetpack Compose)
   Interfaces fluidas y adaptadas al sistema
3. **Seguridad**
   Integración con APIs nativas
   Mejor control de permisos y uso de biometría

4. **Acceso a hardware**
   Uso directo de cámara, GPS y sensores
   Permite funcionalidades avanzadas
5. **Estabilidad y soporte**
   Tecnologías oficiales de Google y Apple
   Mejor soporte y mantenimiento a largo plazo

Desventajas

1. **Costo de desarrollo**
   Se requieren dos aplicaciones (iOS y Android)
   Mayor inversión de tiempo y recursos
2. **Especialización**
   Necesidad de conocimientos en Swift y Kotlin
   Equipos separados por plataforma
3. **Mantenimiento**
   Cambios deben hacerse en ambas apps
   Mayor esfuerzo y riesgo de errores
4. **Tiempo de desarrollo**
   Desarrollo paralelo en dos plataformas
   Entregas más lentas
5. **Complejidad arquitectónica**
   Se manejan arquitecturas separadas
   Mayor esfuerzo en diseño, testing y coordinación

Relevancia actual

1. **Opción premium en la industria**
   Ofrece máximo rendimiento y mejor experiencia de usuario
   Acceso completo al hardware del dispositivo
2. **Seguridad en aplicaciones críticas**
   Permite uso de biometría, cifrado y control de permisos
   Mayor protección de datos sensibles
3. **Tecnologías emergentes**
   Fundamental para AR, IoT, pagos móviles y wearables
   Requiere baja latencia y acceso directo al hardware

4. **Rendimiento y eficiencia**
   Mejor uso de CPU, memoria y batería
   Experiencia más rápida y fluida
5. **Convive con multiplataforma**
   Se usa junto a Flutter o React Native
   Preferido en apps complejas o de alto nivel
6. **Base de innovación**
   Nuevas tecnologías (IA, personalización, seguridad) se integran primero en apps nativas

2. La Era del WebView / Híbridos (Cordova, PhoneGap, Ionic inicial)

¿Qué es?

La Era del WebView / Híbridos fue una etapa del desarrollo móvil en la que se buscó crear aplicaciones usando tecnologías web en lugar de código nativo. La Era del WebView / Híbridos fue una etapa del desarrollo móvil en la que se buscó crear aplicaciones usando tecnologías web en lugar de código nativo.

Tecnologías

- HTML → para la estructura de la aplicación
- CSS → para el diseño y estilos
- JavaScript → para la lógica y funcionalidad

Frameworks y herramientas:

- Apache Cordova
- PhoneGap
- Ionic

Tecnología clave:

- WebView → navegador integrado dentro de la app que muestra el contenido web

Arquitectura

1. Capa de interfaz (UI)

- Desarrollada con **HTML, CSS y JavaScript**
- Es lo que el usuario ve (como una página web

2. WebView (contenedor)

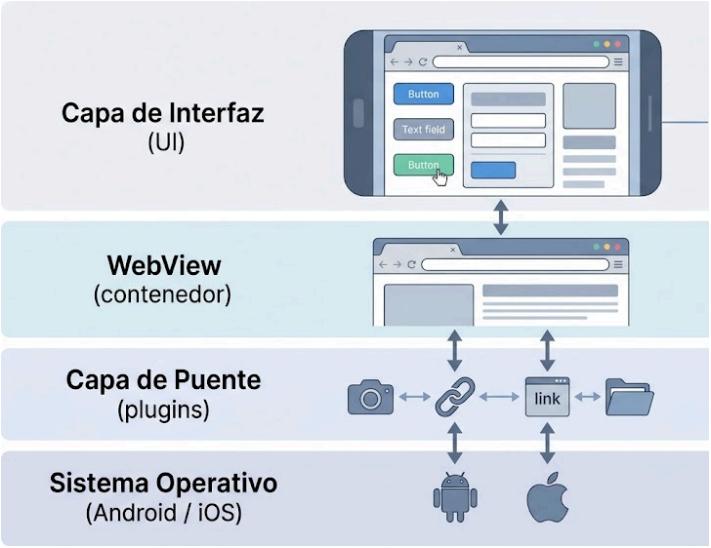
- Es un navegador embebido dentro de la app
- Renderiza la interfaz web
- Actúa como intermediario entre la UI y el sistema operativo

3. Capa de puente (plugins)

- Proporcionada por frameworks como Apache Cordova o PhoneGap
- Permite que JavaScript acceda a funciones nativas:
  - Cámara
  - GPS
  - Archivos

4. Sistema operativo (Android / iOS)

- Ejecuta la aplicación
- Proporciona acceso real al hardware



Implicaciones ingenieriles

1. Rendimiento

- Menor rendimiento que las apps nativas
- Puede ser lento en animaciones o procesos complejos

- Depende del navegador interno (WebView)

2. Experiencia de usuario (UX)

- Puede sentirse menos fluida
- Interfaz menos "natural" comparada con apps nativas
- Posibles retrasos en la interacción

3. Desarrollo

- Más rápido y económico
- Un solo código funciona en varias plataformas (Android y iOS)
- Usa tecnologías conocidas (HTML, CSS, JavaScript)

4. Acceso a hardware

- Limitado y depende de plugins
- Algunas funciones no están disponibles o funcionan peor
- Dependencia de herramientas como Apache Cordova

5. Mantenimiento

- Más fácil de actualizar
- Pero dependes de librerías externas y plugins

Ventajas

1. Desarrollo rápido
   Permite crear aplicaciones en menos tiempo usando tecnologías web conocidas como HTML, CSS y JavaScript.
2. Multiplataforma
   Un solo código funciona en Android y iOS, evitando desarrollar dos aplicaciones separadas.
3. Menor costo
   Reduce costos de desarrollo y mantenimiento al no necesitar equipos distintos para cada sistema operativo.
4. Facilidad de aprendizaje
   Los desarrolladores web pueden crear apps móviles sin aprender lenguajes nativos como Kotlin o Swift.

5. Mantenimiento sencillo
   Es más fácil actualizar la aplicación, ya que muchos cambios se hacen directamente en el código web.
6. Acceso a funciones del dispositivo
   A través de herramientas como Apache Cordova, se puede acceder a cámara, GPS y otros recursos mediante plugins.

Desventajas

1. Bajo rendimiento
   Son más lentas que las aplicaciones nativas, especialmente en animaciones, gráficos o procesos complejos.
2. Experiencia de usuario limitada
   La interfaz puede sentirse menos fluida y menos "natural" en comparación con apps nativas.
3. Dependencia del WebView
   El rendimiento depende del navegador interno del dispositivo, lo que puede variar entre equipos.
4. Acceso limitado al hardware
   No todas las funciones del dispositivo están disponibles o funcionan correctamente.
   Depende de plugins como Apache Cordova.
5. Problemas con plugins
   Los plugins pueden fallar, estar desactualizados o no ser compatibles con todas las versiones del sistema.
6. Menor escalabilidad
   No es la mejor opción para aplicaciones grandes, complejas o de alto rendimiento.

Relevancia actual

Actualmente, el desarrollo híbrido basado en WebView tiene una relevancia media-baja, ya que ha sido reemplazado en gran parte por tecnologías más modernas, aunque aún se utiliza en aplicaciones simples por su rapidez y bajo costo.

3. El Puente Nativo-JavaScript y otros puentes

¿Qué es?

El Puente Nativo–JavaScript (Native–JavaScript Bridge) es un mecanismo de comunicación que permite que el código escrito en JavaScript interactúe con funcionalidades nativas del

sistema operativo (como cámara, GPS o sensores), actuando como una capa intermedia entre el entorno JavaScript y el código nativo.

Tecnologías

- **Motores de JavaScript**
  Ejecutan la lógica de la app en JS
  Ejemplos: JavaScriptCore, V8, Hermes
- **Bridge (comunicación)**
  Permite la comunicación entre JS y código nativo
  Usa mensajes (JSON) y llamadas asíncronas
- **Módulos o plugins nativos**
  Acceden a funciones del dispositivo (cámara, GPS, sensores)
  Actúan como intermediarios entre JS y el sistema
- **WebView**
  Ejecuta HTML, CSS y JS dentro de la app
  Se comunica con el código nativo mediante el bridge
- **APIs nativas del sistema**
  Android SDK e iOS SDK
  Ejecutan acciones reales sobre hardware y sistema
- **Nuevas tecnologías (sin bridge)**
  - JSI (JavaScript Interface)
    Permite comunicación directa JS–Nativo con menor latencia

Arquitectura

- **Capa JavaScript**
  Ejecuta la lógica de la aplicación
  Genera solicitudes hacia el sistema nativo
- **Capa Bridge (comunicación)**
  Intermediario entre JS y código nativo
  Usa mensajes y comunicación asíncrona (JSON)
  Puede generar latencia
- **Capa de módulos nativos**
  Implementada en Java/Kotlin (Android) o Swift (iOS)
  Ejecuta funciones como cámara, GPS y sensores
- **Capa del sistema operativo**
  Android / iOS
  Ejecuta las APIs reales y controla el hardware
- **Flujo de ejecución**

Usuario → JavaScript → Bridge → Módulo nativo → Sistema operativo → respuesta → Bridge → JavaScript → UI

Implicaciones ingenieriles

Control total sobre:

- memoria
- hilos (threads)
- renderizado

Permite optimización a bajo nivel

Ventajas

1. Máximo rendimiento (sin overhead)
2. Latencia mínima
3. Acceso inmediato a nuevas APIs

Desventajas

1. Overhead del bridge
2. complejidad en debugging
3. dependencia de plugins nativos

Relevancia actual

1. Evolución tecnológica: El bridge tradicional ha sido reemplazado en gran parte por JSI, Fabric y TurboModules, que mejoran la comunicación entre JavaScript y código nativo.

2. Uso actual: Sigue siendo clave en frameworks como React Native, permitiendo desarrollo multiplataforma.

3. Importancia en la industria: Se mantiene vigente porque reduce costos y acelera el desarrollo de apps.

4. Mejora de rendimiento: Las nuevas arquitecturas reducen la latencia y los problemas del bridge clásico.

5. Valor general: Es una solución en evolución que equilibra productividad y rendimiento.

4. El Motor de Renderizado Propio

¿Qué es?

El Motor de Renderizado Propio es un enfoque de desarrollo móvil en el que la aplicación no usa los componentes visuales nativos del sistema operativo, sino que dibuja toda la interfaz desde cero utilizando su propio motor gráfico.

Tecnologías

Framework principal:

- Flutter

Lenguaje de programación:

- Dart

Motor gráfico:

- Skia (motor de renderizado que dibuja la interfaz)

Componentes clave:

- Widgets (elementos de la interfaz)
- Renderizado en canvas (dibujo directo en pantalla)

Soporte multiplataforma:

- Android
- iOS
- Web
- Desktop

Arquitectura

1. Capa de aplicación (lógica)

- Escrita en Dart usando Flutter
- Define la lógica y la estructura de la app

2. Capa de widgets

- Todo se construye con widgets (botones, textos, layouts)
- Describe cómo debe verse la interfaz

3. Motor de renderizado

- Usa Skia para dibujar la interfaz
- Renderizar directamente en un lienzo (canvas)
- No usa componentes nativos

4. Plataforma (Android / iOS)

- Solo se encarga de ejecutar la app
- Proporciona acceso a hardware mediante canales nativos

Implicaciones ingenieriles

1. Mayor control sobre la interfaz
El uso de frameworks como Flutter permite controlar completamente cómo se dibuja cada elemento, sin depender del sistema operativo.

2. Alto rendimiento gráfico
Al usar un motor como Skia, el renderizado es rápido y consistente, ideal para animaciones y transiciones fluidas.

3. Consistencia multiplataforma
La interfaz se ve igual en Android, iOS y otras plataformas, evitando problemas de diseño entre sistemas.

4. Mayor consumo de recursos
Al dibujar toda la interfaz desde cero, puede consumir más memoria y procesamiento que una app nativa en ciertos casos.

5. Menor dependencia del sistema operativo

No depende de componentes nativos, lo que reduce problemas de compatibilidad entre versiones del sistema.

6. Curva de aprendizaje

Requiere aprender nuevas tecnologías como Dart y la arquitectura de widgets.

7. Integración con hardware

El acceso a funciones nativas se hace mediante canales (bridges), lo que puede requerir código adicional.

Ventajas

1. Alto rendimiento
   Gracias a su motor gráfico (como el usado en Flutter), permite animaciones fluidas y mejor desempeño.
2. Consistencia visual
3. La aplicación se ve igual en todas las plataformas (Android, iOS, web), sin depender del sistema operativo.
4. Control total del diseño
   El desarrollador tiene control completo sobre la interfaz, pudiendo personalizar cada detalle.
5. Multiplataforma real
   Un solo código funciona en múltiples sistemas operativos, reduciendo tiempo y esfuerzo.
6. Menor dependencia del sistema
   No depende de componentes nativos, evitando problemas de compatibilidad entre versiones.
7. Mejor experiencia de usuario
   Ofrece interfaces más modernas, rápidas y uniformes.

Desventajas

1. Mayor consumo de recursos
   Al dibujar toda la interfaz desde cero, puede usar más memoria y procesamiento en comparación con apps nativas.
2. Tamaño de aplicación más grande
   Las apps suelen pesar más porque incluyen su propio motor gráfico (como en Flutter).
3. Curva de aprendizaje
   Requiere aprender nuevas tecnologías como Dart y la arquitectura basada en widgets.

4. Integración con funciones nativas
   El acceso al hardware (cámara, GPS, etc.) necesita canales o código adicional, lo que puede complicar el desarrollo.
5. Dependencia del framework
   La aplicación depende completamente del framework; si este tiene problemas o cambios, puede afectar el proyecto.
6. No ideal para todos los casos
   Para aplicaciones muy específicas del sistema operativo, el desarrollo nativo puede ser más eficiente.

Relevancia actual

Actualmente, el motor de renderizado propio tiene una alta relevancia, ya que permite desarrollar aplicaciones modernas, rápidas y multiplataforma, siendo una de las principales tendencias en el desarrollo móvil actual.

5. El Futuro: Foldables, AR y Wearables

¿Qué es?

Es un paradigma de desarrollo de software móvil basado en interfaces adaptativas, computación espacial y dispositivos conectados, que permite la interacción del usuario en múltiples entornos físicos y digitales.

Tecnologías

1. Foldables (pantallas plegables)

- Android Jetpack WindowManager
- Responsive/adaptive UI design
- Material Design adaptable

2. Realidad Aumentada (AR)

- ARCore
- ARKit

3. Wearables

- Wear OS

- watchOS

Arquitectura interna

1. Capa de Interfaz Adaptativa

- UI flexible según pantalla o dispositivo
- Uso de layouts responsivos

2. Capa de Sensores

- Cámara
- Movimiento
- Biometría
- Profundidad (AR)

3. Capa de Contexto

- Detecta entorno del usuario

4. Capa de Procesamiento

- Renderizado 3D (AR)
- IA en tiempo real
- Procesamiento local (edge computing)

5. Capa del Sistema Operativo

- Android
- iOS

Implicaciones ingenieriles

- Foldables: permiten que las aplicaciones se adapten a pantallas que se pueden doblar, mejorando la experiencia en modo teléfono o tablet.
- Realidad Aumentada (AR): ya se usa en juegos, mapas y compras, combinando elementos digitales con el mundo real para hacer las apps más interactivas.
- Wearables: como relojes inteligentes, permiten monitorear la salud, actividad física y recibir notificaciones en tiempo real.

Ventajas

1. Experiencias inmersivas (AR)
2. Interacción más natural
3. Mayor movilidad del usuario
4. Interfaces adaptativas (foldables)
5. Monitoreo de salud en tiempo real (wearables)
6. Nuevas formas de interacción humano-computadora

Desventajas

1. Alta complejidad de desarrollo
2. Mayor costo de implementación
3. Consumo de batería elevado (AR y wearables)
4. Fragmentación de dispositivos
5. Difícil testing en escenarios reales
6. Dependencia fuerte del hardware

Relevancia actual

1. Smartphones plegables en expansión
2. AR en educación, medicina y comercio
3. Wearables enfocados en salud digital
4. Ecosistemas como realidad mixta (XR)

Parte 2: Taller Práctico - Análisis Crítico de Aplicaciones

A continuación, formen equipos de trabajo. Vamos a analizar dos aplicaciones reales y altamente escalables que tomaron decisiones tecnológicas distintas.

Caso de Estudio A: Temu (Tecnologías tipo React y webviews)

Caso de Estudio B: Bitwarden (.NET MAUI, C# y Xamarin)

Instrucciones para el equipo:

Deben debatir, investigar (pueden usar sus equipos) y redactar un reporte crítico respondiendo las siguientes interrogantes. No busquen respuestas "correctas", busquen respuestas fundamentadas con ingeniería.

- ***Ingeniería inversa de la decisión tecnológica***

Al analizar el caso de Temu, es importante entender que su naturaleza como plataforma de comercio electrónico masivo condiciona profundamente sus decisiones tecnológicas. No se trata simplemente de construir una aplicación rápida, sino de sostener un modelo de negocio basado en iteración constante, personalización agresiva y escalabilidad global. Bajo esta lógica, resulta razonable inferir que Temu ha optado por una arquitectura híbrida, donde conviven componentes nativos con capas web dinámicas (como WebViews o tecnologías tipo React). Esta decisión no responde a una búsqueda de rendimiento máximo, sino a la necesidad de reducir el tiempo entre la concepción de una funcionalidad y su despliegue en producción.

Desde una perspectiva de ingeniería, esta elección implica aceptar una pérdida de eficiencia en la ejecución especialmente en renderizado y manejo de eventos a cambio de una ganancia significativa en flexibilidad. La capacidad de modificar interfaces, lanzar experimentos A/B y adaptar el contenido sin pasar por los ciclos de aprobación de las tiendas móviles representa una ventaja competitiva crítica. En este sentido, Temu prioriza la velocidad del negocio sobre la pureza técnica, lo cual es coherente con su contexto.

Por otro lado, el caso de Bitwarden responde a un problema radicalmente distinto. Al tratarse de un gestor de contraseñas, su principal responsabilidad no es la interacción visual ni la velocidad de iteración, sino la protección de información altamente sensible. Bitwarden utiliza un stack basado en C# y .NET (históricamente Xamarin y más recientemente .NET MAUI), lo cual le permite compartir lógica crítica entre plataformas sin sacrificar control sobre el comportamiento del sistema.

Esta decisión está profundamente alineada con principios de seguridad y mantenibilidad. Centralizar la lógica de cifrado en un solo lenguaje reduce la superficie de error y facilita auditorías, algo fundamental en software donde una vulnerabilidad puede comprometer datos personales de millones de usuarios. A diferencia de Temu, Bitwarden no puede permitirse inconsistencias entre plataformas ni depender de capas intermedias que introduzcan incertidumbre en la ejecución. Aquí, la ingeniería no busca velocidad de cambio, sino previsibilidad, trazabilidad y control absoluto del flujo de datos.

- ***Análisis de rendimiento y arquitectura***

En términos de arquitectura, Bitwarden enfrenta desafíos particularmente complejos debido a la necesidad de combinar una interfaz multiplataforma con operaciones criptográficas intensivas. El uso de .NET MAUI o Xamarin implica que la interfaz de usuario no siempre es completamente nativa, lo que puede introducir cierta sobrecarga en el renderizado. Sin embargo, el verdadero reto no está en la UI, sino en garantizar que las operaciones críticas como el cifrado y descifrado de contraseñas se ejecuten de manera eficiente y segura.

Esto obliga a diseñar cuidadosamente la gestión de hilos (multithreading), evitando que procesos pesados bloqueen la interfaz. Además, la integración con componentes nativos como el almacenamiento seguro del sistema (Keychain en iOS o Keystore en Android) introduce una capa adicional de complejidad, ya que se debe garantizar que las claves nunca abandonen entornos protegidos. En este contexto, la arquitectura de Bitwarden no solo debe ser eficiente, sino también resistente a ataques y errores de implementación.

En contraste, Temu enfrenta un tipo de problema completamente diferente. Su principal desafío no es la seguridad extrema, sino el rendimiento percibido en escenarios de alto volumen de datos, como listas de productos o feeds infinitos. Si consideramos que parte de su interfaz puede estar basada en tecnologías web o capas híbridas, el principal cuello de botella surge en la gestión del renderizado y la comunicación entre capas.

El problema clásico en este tipo de arquitecturas es la sobrecarga al manejar grandes cantidades de elementos visuales, especialmente cuando se combinan imágenes, animaciones y eventos de usuario. Para mantener una experiencia fluida, Temu debe apoyarse en técnicas como la virtualización de listas, donde solo se renderizan los elementos visibles en pantalla, así

como en estrategias agresivas de caching y distribución de contenido mediante CDNs. Desde la ingeniería, esto implica trasladar parte de la complejidad al backend y a la infraestructura, compensando así las limitaciones del frontend híbrido.

- ***El desafío de nuevas plataformas (Realidad Aumentada)***

Si se introduce el escenario hipotético de nuevas plataformas como dispositivos de realidad aumentada, las limitaciones de ambas arquitecturas se vuelven evidentes. Temu, al depender parcialmente de tecnologías web o híbridas, enfrentaría serias dificultades para adaptarse a entornos donde el rendimiento gráfico y la interacción en tiempo real son críticos. Las WebViews y capas abstractas no están diseñadas para manejar renderizado 3D complejo ni para interactuar directamente con sensores avanzados.

Por su parte, Bitwarden, aunque tampoco está orientado a este tipo de entornos, tendría una ligera ventaja debido a su mayor cercanía con el código nativo. Su arquitectura basada en .NET permite integrar módulos específicos que interactúen directamente con APIs del sistema, lo cual facilita una eventual adaptación. Sin embargo, desde una perspectiva estrictamente técnica, ninguna de las dos soluciones está diseñada para este tipo de escenarios.

Esto lleva a una conclusión importante: cuando el paradigma de interacción cambia (por ejemplo, de pantallas táctiles a entornos inmersivos), las abstracciones multiplataforma tienden a romperse. En estos casos, la ingeniería vuelve a apoyarse en soluciones nativas o motores especializados.

- ***Veredicto crítico desde la ingeniería***

Al plantear el escenario de desarrollar una aplicación que combine mapas 3D interactivos, notificaciones de alta frecuencia y soporte para dispositivos plegables, la discusión se traslada a los límites reales de cada enfoque tecnológico. Ni la arquitectura híbrida de Temu ni el enfoque multiplataforma de Bitwarden están diseñados para maximizar el rendimiento en contextos de alta exigencia gráfica y control del hardware.

El desarrollo nativo (usando lenguajes como Swift o Kotlin) ofrece ventajas difíciles de igualar. El acceso directo a la GPU, la integración inmediata con nuevas APIs del sistema y la ausencia de capas intermedias permiten reducir la latencia y maximizar la eficiencia. Además, en escenarios donde la interacción con el hardware es crítica como sensores, cámaras o procesamiento en tiempo real cualquier capa de abstracción se convierte en un posible punto de falla o degradación del rendimiento.

Por tanto, la decisión de optar por desarrollo nativo no responde a una preferencia subjetiva, sino a una evaluación objetiva de las necesidades del sistema. Mientras que Temu y Bitwarden

son ejemplos de cómo diferentes contextos justifican diferentes arquitecturas, ambos también evidencian que, en escenarios de máxima exigencia técnica, el control directo sobre el hardware sigue siendo el factor determinante.

- ***Conclusión general***

El contraste entre Temu y Bitwarden ilustra cómo las decisiones tecnológicas no pueden evaluarse de forma aislada, sino en función del problema que buscan resolver. Temu demuestra que, en entornos donde la velocidad de innovación es clave, es válido sacrificar eficiencia en favor de flexibilidad. Bitwarden, en cambio, evidencia que cuando la seguridad es el eje central, la arquitectura debe priorizar control, consistencia y auditabilidad.

Desde la ingeniería, ninguna decisión es universalmente correcta. Sin embargo, el análisis revela un patrón claro: cuanto más crítico es el rendimiento, la seguridad o la interacción con el hardware, menor margen existe para utilizar abstracciones multiplataforma. En ese punto, la ingeniería vuelve a sus fundamentos: control, eficiencia y cercanía al sistema.

- ***¿Qué es un aplicativo nativo, hay varios, como se clasifican?***

Un aplicativo nativo es una aplicación desarrollada específicamente para un sistema operativo, utilizando sus lenguajes y herramientas oficiales. Esto le permite integrarse directamente con el sistema y el hardware, logrando mejor rendimiento y acceso completo a funcionalidades como cámara, GPS o sensores. Por ejemplo, apps para Android o iOS.

Se pueden clasificar principalmente de tres formas:

- Según la plataforma: Android, iOS, Windows, etc.
- Según el dispositivo: móviles, escritorio, wearables.
- Según su propósito: alto rendimiento (juegos), aplicaciones críticas (banca, seguridad), o uso general.

En resumen, lo que define a un aplicativo nativo es su desarrollo específico para una plataforma, lo que garantiza mayor eficiencia y control.

Referencias Bibliográficas:

- Google Developers. (s.f.). Performance Best Practices. https://developer.android.com/topic/performance
- Data.ai. (s.f.). *Mobile App Performance Reports*. https://www.data.ai
- Bitwarden. (s.f.). *Security Whitepaper*. https://bitwarden.com/help/security-white-paper/
- Bitwarden Inc.. (s.f.). *Bitwarden Open Source Repository*. https://github.com/bitwarden
- Chat GPT. Open AI.